

Martwily FH. Simelutung
2505281294

Divide and conquer

1. Membagi masalah ke bagian kecil
2. menyelesaikan masalah per bagian secara recursive
3. menggabungkan penyelesaian per bagian menjadi solusi masalah awal

Studi kasus : MCSS

* diberikan angka int negatif dimungkinkan A_1, A_2, A_N cari nilai max dari $(A_i + A_{i+1} + \dots + A_j)$

MCSS adalah nd jika semua int adalah negatif

* membagi permasalahan menjadi lebih kecil

- deretan bilangan input di bagi dua menjadi dua bagian yg

masing² lebih kecil dari input awal

- identifikasi kemungkinan yg terjadi

* Menyelesaikan (conquer) masalah per bagian secara recursive

- lakukan pemanggilan rekursif seperti trap² bagian

* Menggabungkan penyelesaian trap bagian menjadi penyelesaian awal
- bandingkan hasil trap kemungkinan

Dari langkah tersebut menghasilkan runtime $O(N \log N)$

Binary Search

CD diberikan angka search value yg dicari dan array yg terurut
arr[], cari index i dimana arr[i] $\geq$ search value

Contoh : mencari angka 33

Awalnya : mencari dari index 0(10) sampai 14(hi)

Binary search akan melakukan loop dengan cara ~~array~~ mengurutkan array, mencari nilai tengah, melihat apakah angka dicari kurang dari atau lebih dari angka tengah. Jika kurang dan more array akan fokus ke kiri array tengah begitu pula sebaliknya ~~dan~~ dan melakukan looping pada langkah / hingga mendapatkan angka

Maxwell F.H. Suturpan
2506584294

Dynamic programming : Sebuah pendekatan mencari solusi masalah dengan memecahkan masalah awal menjadi masalah $\Rightarrow$ masalah yg lebih kecil
Dip dapat diterapkan pd masalah yg memiliki sifat:

- * Overlapping sub-problems : terdapat sub-problems yg sama persis
- * Optimal substructure : Solusi yg optimal dapat dibentuk dari solusi optimal dari sub-problemsnya

Case study : Change-making

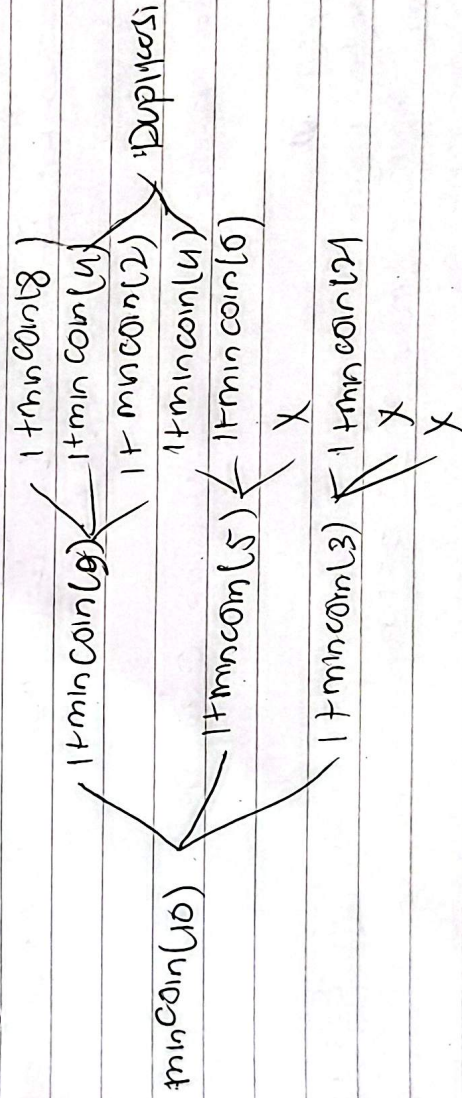
den coin yg tersedia $c_1, c_2, \dots, c_n$ (cents) tentukan dlm minimum coin yg diperlukan utk "kombinasi" sejumlah cents

Analisa masalah

Misalkan yg ada hanya 3 coin saja

Coin = 1, 5, 7

Change = 10



Algoritma greedy

CD mengambil keputusan terbaik pada saat (locally optimal choice) itu dan berharap dapat memperoleh solusi terbaik (global optimum)

* Ambil kon terbesar yg tersedia secara berulang-ulang

* dgn coin : 1, 5, 10 dan 25 cent, kembalikan 63 cent : 25, 25, 10, 3

1, 1, 1

* Solusi tidak selalu yg terbaik, hanya optimum lokal

Contoh: ada koin 1 dan 3 cent, maka hasil algoritma greedy seperti diatas adalah 10 dan 3 cent, yg lebih optimum yg 3 coin (3 koin 21)

Mawully F.H. Sinetung
~~2506181294~~ 2506181294

Dynamic Programming

CO bertujuan utk memecahkan setiap sub-Problem hanya satu kali
akibanya mengurangi banyaknya komputasi yg perlu dilakukan

Backtracking algoritma

CO cara sena kemungkinan penyelesaian jika hasilnya tidak
memuaskan kembali ke tahap sebelumnya dan cara kemungkinan lain.
Proses berhenti jika ditemukan

* Pruning : menghilangkan subset set kemungkinan supaya proses lebih
efisien